Step 1: Importing the Loader

In [1]:
```
from langchain_community.document_loaders import PyPDFLoader
```

Step 2: Load Documents

Reading PDF files using PyPDF

In [2]:
```
from langchain_community.document_loaders import DirectoryLoader, PyPDFLoader

loader = DirectoryLoader(
    "data",
    glob="*.pdf",
    loader_cls=PyPDFLoader
)

documents = loader.load()

print(len(documents))
```

```
1104
```

Step 3: Cleaning Text

Remove unwanted characters and spaces

In [3]:
```
cleaned_documents = []

for doc in documents:
    text = doc.page_content

    # إزالة الأسطر الفاضية
    text = text.replace("\n", " ")

    # إزالة المسافات الزائدة
    text = " ".join(text.split())

    # تحديث النص
    doc.page_content = text

    cleaned_documents.append(doc)

print(len(cleaned_documents))
```

```
1104
```

Step 4: Text Splitting

Split cleaned documents into chunks

In [4]:
```
from langchain_text_splitters import RecursiveCharacterTextSplitter

large_splitter = RecursiveCharacterTextSplitter(
    chunk_size=1000,
    chunk_overlap=200
)

large_chunks = large_splitter.split_documents(cleaned_documents)

print(f"Total Chunks Created: {len(large_chunks)}\n")

print("--- Sample Large Chunk ---")
print(large_chunks[0].page_content)
```

```
Total Chunks Created: 5578

--- Sample Large Chunk ---
A Deep Learning-Based Time Shift Objective Function for Full Waveform Inversion Item
Type Article Authors Alfarhan, Mustafa;Chen, Fuqiang;Turkiyyah, George;Keyes, David
E.;Vasconcelos, Ivan;Ravasi, Matteo Citation Alfarhan, M., Chen, F., Turkiyyah, G.,
Keyes, D., Vasconcelos, I., & Ravasi, M. (2026). A Deep Learning-Based Time Shift Ob
jective Function for Full Waveform Inversion. Geophysical Prospecting, 74(3). Portic
o. https://doi.org/10.1111/1365-2478.70160 Eprint version Publisher's Version/PDF DO
I 10.1111/1365-2478.70160 Publisher Wiley Journal Geophysical Prospecting Rights Arc
hived with thanks to Geophysical Prospecting under a Creative Commons license, detai
ls at: http://creativecommons.org/ licenses/by/4.0/ Download date 2026-04-11 17:08:4
8 Item License http://creativecommons.org/licenses/by/4.0/ Link to Item http://hdl.h
andle.net/10754/709123
```

Step 5: Create Embeddings

Using BGE model for better semantic understanding

In [5]:
```
from langchain_huggingface import HuggingFaceEmbeddings

print("Loading Embedding Model...")

embedding_model = HuggingFaceEmbeddings(
    model_name="BAAI/bge-base-en-v1.5",
    encode_kwargs={'normalize_embeddings': True}
)

print("Model Successfully Loaded!")
```

```
Loading Embedding Model...
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOK
EN to enable higher rate limits and faster downloads.
Loading weights:   0%|          | 0/199 [00:00<?, ?it/s]
Model Successfully Loaded!
```

In [6]:
```
print(len(large_chunks))
```

```
5578
```

Step 6: Creating and Persisting the Vector Database

Store embeddings in ChromaDB for efficient retrieval

In [7]:
```
# Step 6: Creating and Persisting the Vector Database (Fixed)

from langchain_community.vectorstores import Chroma

persist_dir = "vector_DB"

print("Building Vector Database...")

#  DB  الأول
vector_store = Chroma(
    persist_directory=persist_dir,
    embedding_function=embedding_model
)

# حجم كل batch
batch_size = 500

# إضافة البيانات على دفعات
for i in range(0, len(large_chunks), batch_size):
    batch = large_chunks[i:i+batch_size]
    vector_store.add_documents(batch)
    print(f"Added batch {i//batch_size + 1}")

print(f"✅ Vector Database successfully created and saved in folder: '{persist_dir}
```

```
Building Vector Database...
C:\Users\W.I\AppData\Local\Temp\ipykernel_8056\3688377182.py:10: LangChainDeprecatio
nWarning: The class `Chroma` was deprecated in LangChain 0.2.9 and will be removed i
n 1.0. An updated version of the class exists in the `langchain-chroma package and s
hould be used instead. To use it run `pip install -U `langchain-chroma` and import a
s `from `langchain_chroma import Chroma``.
  vector_store = Chroma(
```

```
Added batch 1
Added batch 2
Added batch 3
Added batch 4
Added batch 5
Added batch 6
Added batch 7
Added batch 8
Added batch 9
Added batch 10
Added batch 11
Added batch 12
✅ Vector Database successfully created and saved in folder: 'vector_DB'
```

Step 7: Initialize Retriever

Retrieve relevant chunks from the vector database

In [8]:
```
# إنشاء الـ Retriever من قاعدة البيانات
retriever = vector_store.as_retriever(
    search_type="mmr",   # علشان يجيب نتائج متنوعة
    search_kwargs={'k': 4}  # يرجع أفضل 4 أجزاء
)

print("Retriever is ready!")
```

```
Retriever is ready!
```

Step 8: Initialize Gemini API

In [9]:
```
import os
from langchain_google_genai import ChatGoogleGenerativeAI

# ـ API Key
os.environ["GOOGLE_API_KEY"] = "AIzaSyC9cqPT8-97H2zyhdHmAkVohTQouDQuEQE"

# Initialize Gemini
llm = ChatGoogleGenerativeAI(
    model="gemini-1.5-flash",
    temperature=0.2
)

print("Gemini API is connected!")
```

```
Gemini API is connected!
```

Step 9: Create Prompt Template

Guide the AI to answer based on retrieved documents

In [10]:
```
from langchain_core.prompts import ChatPromptTemplate

# System Prompt (تعليمات للـ AI)
system_prompt = (
    "You are a helpful academic assistant specialized in Computer, Electrical, and
    "Answer the user's question based only on the provided context from research ar
    "If the answer is not in the context, say 'I don't know based on the provided d
    "\n\n"
    "Context: {context}"
)

# Create Prompt Template
prompt = ChatPromptTemplate.from_messages([
    ("system", system_prompt),
    ("human", "{input}"),
])
```

Step 10: Build RAG Pipeline (Modern Approach)

Link Retriever, Prompt, and LLM together

In [11]:
```
# Define function to format retrieved documents
def format_docs(docs):
    return "\n\n".join(doc.page_content for doc in docs)

# Create RAG pipeline manually (بدون chains القديمة)
rag_chain = (
    {
        "context": retriever | format_docs,
        "input": lambda x: x["input"]
    }
    | prompt
    | llm
)
```

In [ ]:
```
what is the metabolic insights

ماذا تعرف عن ال e-learning

ماذا تعرف عنMetabolomic insights into drought tolerance mechanisms

what do you know about  Application in encapsulated large-area PSTs
```

In [ ]:
```
AIzaSyCdbBqeUdL1glQU_oU9xSR8Hs-IxOOgiwU
```